

Go-ing Fast with MCP

Rapid Development of Custom MCP Servers
for Security Agents



Matt Franz, Pydantic
DC State of the Stack 2026



During the next five minutes you will learn...

Why you should consider writing a custom MCP server for data-intensive platforms (SIEM, EDR, observability)

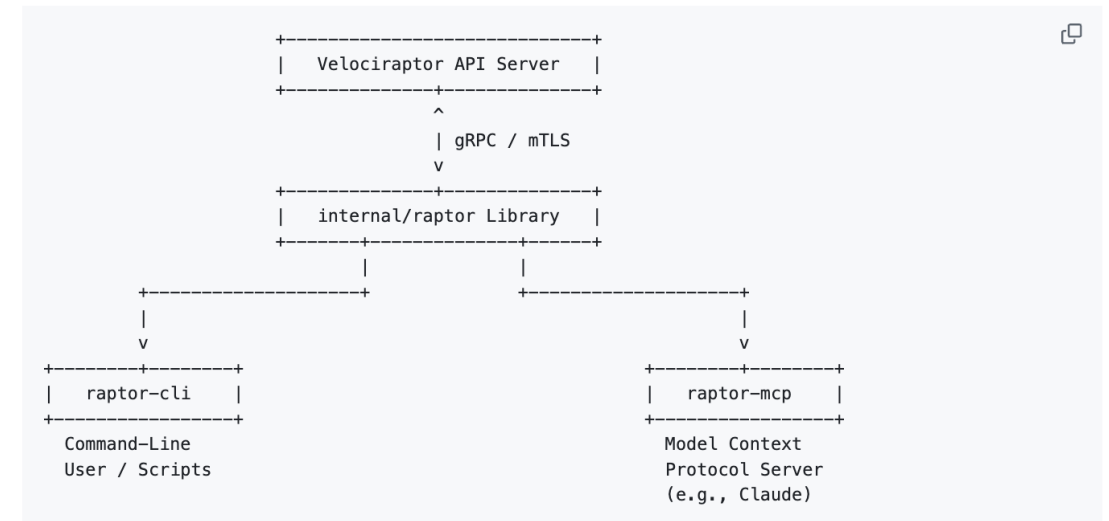
How to *iteratively* build, test, and run STDIO MCP servers (and CLIs) for Python agents or Claude, Codex, OpenCode, and more...

README

go-velociraptor-mcp

A Go-based implementation of a Velociraptor command-line interface (`raptor-cli`) and a Model Context Protocol (MCP) server (`raptor-mcp`) for remote endpoint visibility, forensics, and orchestration via Velociraptor's gRPC/mTLS API.

Architecture Overview



Both binaries are powered by a shared backend package (`internal/raptor`) that manages mTLS configuration parsing, connection pooling, raw VQL queries, parameter marshaling, and safety validation.

Who am I?

2.5 decades in Security and Infra

Security & Compliance Lead at Pydantic

For the last 2.5 years, I've been building blue-team security agents

In 2026, I've built over a dozen MCP servers in Go, Python, and Rust

Love European/Canadian travel and read way too much military history!

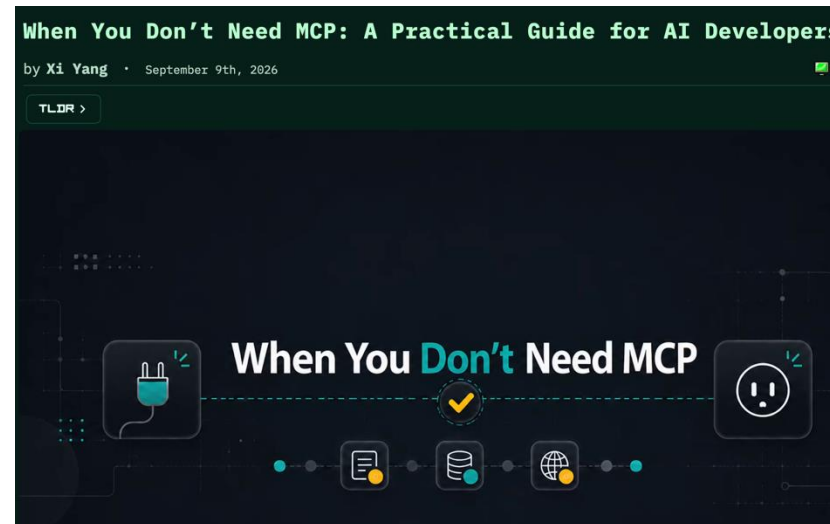


MCP and its Discontents

Token burn — large JSON payloads overwhelming agent context window

Agent thrashing — hallucinated queries and API calls leading to long response delays

(In)security — inherent vulnerabilities, RBAC challenges, and more



Member-only story

AI + Coding + Web Development + Artificial Intelligence + Design +

MCP is Dead

Why you should avoid using MCP in Claude Code and what to use instead

Long Live MCP!

Alternative (complementary) approaches:

- CLIs called by coding agents
- Python code generation – processing and REST APIs

Where I find MCP still adds value:

- Optimized tool calls (and descriptions) a
- Input/output query guardrails
- Easier to package than SKILLS.md

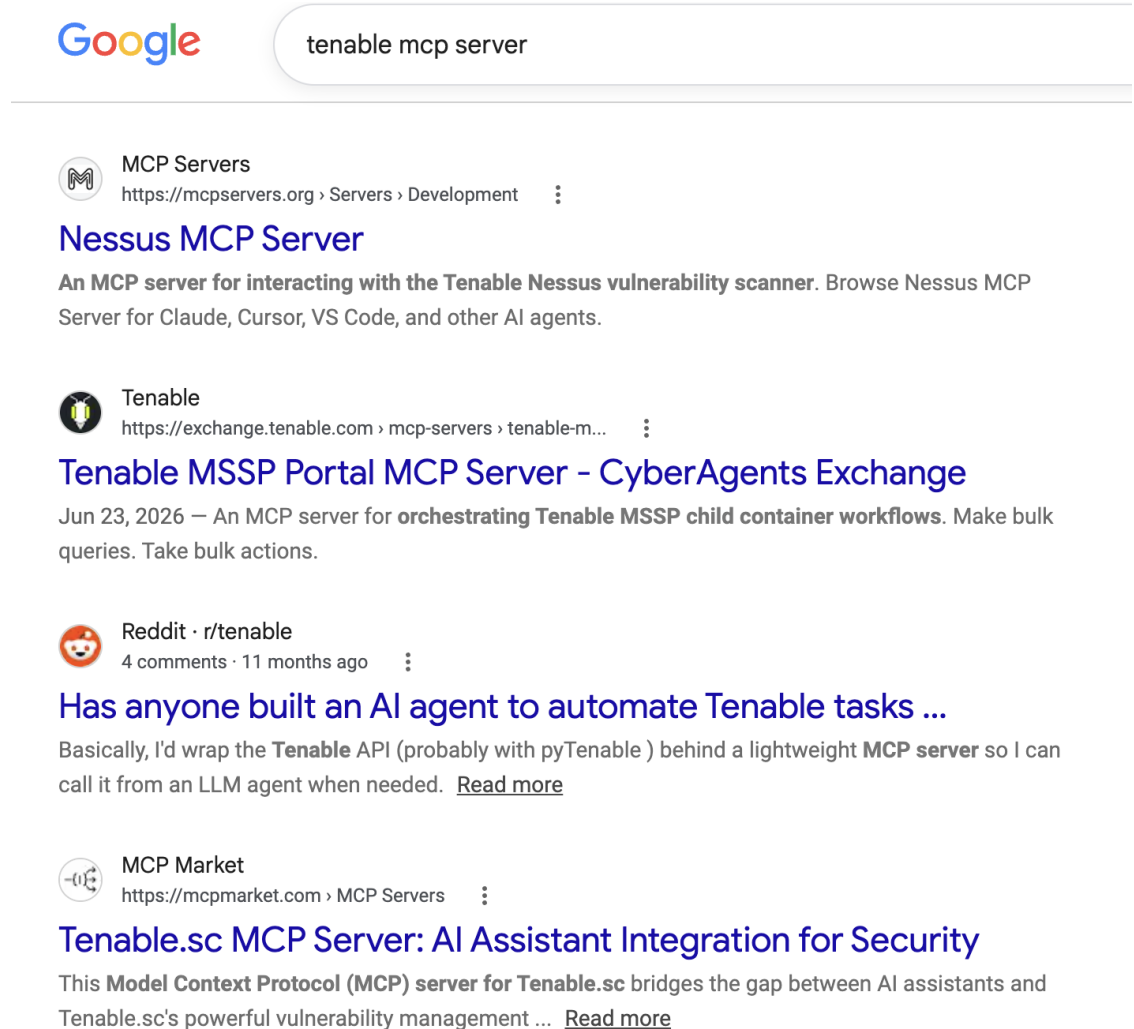


Why build custom MCP servers?

Issues using off the shelf MCP Servers

- Minimal logging or caching — often black boxes.
- Generic tool calls — tuned for everyone, not your telemetry or workflows
- Sketchy MCP marketplaces — do you really trust them with your SIEM API keys?

You probably need to learn the APIs (and data) anyway...



The screenshot shows a Google search interface with the query "tenable mcp server". The search results are as follows:

- MCP Servers**
https://mcpservers.org › Servers › Development
Nessus MCP Server
An MCP server for interacting with the Tenable Nessus vulnerability scanner. Browse Nessus MCP Server for Claude, Cursor, VS Code, and other AI agents.
- Tenable**
https://exchange.tenable.com › mcp-servers › tenable-m...
Tenable MSSP Portal MCP Server - CyberAgents Exchange
Jun 23, 2026 — An MCP server for **orchestrating Tenable MSSP child container workflows**. Make bulk queries. Take bulk actions.
- Reddit · r/tenable**
4 comments · 11 months ago
Has anyone built an AI agent to automate Tenable tasks ...
Basically, I'd wrap the **Tenable** API (probably with pyTenable) behind a lightweight **MCP server** so I can call it from an LLM agent when needed. [Read more](#)
- MCP Market**
https://mcpmarket.com › MCP Servers
Tenable.sc MCP Server: AI Assistant Integration for Security
This **Model Context Protocol (MCP) server for Tenable.sc** bridges the gap between AI assistants and Tenable.sc's powerful vulnerability management ... [Read more](#)

Why Go?

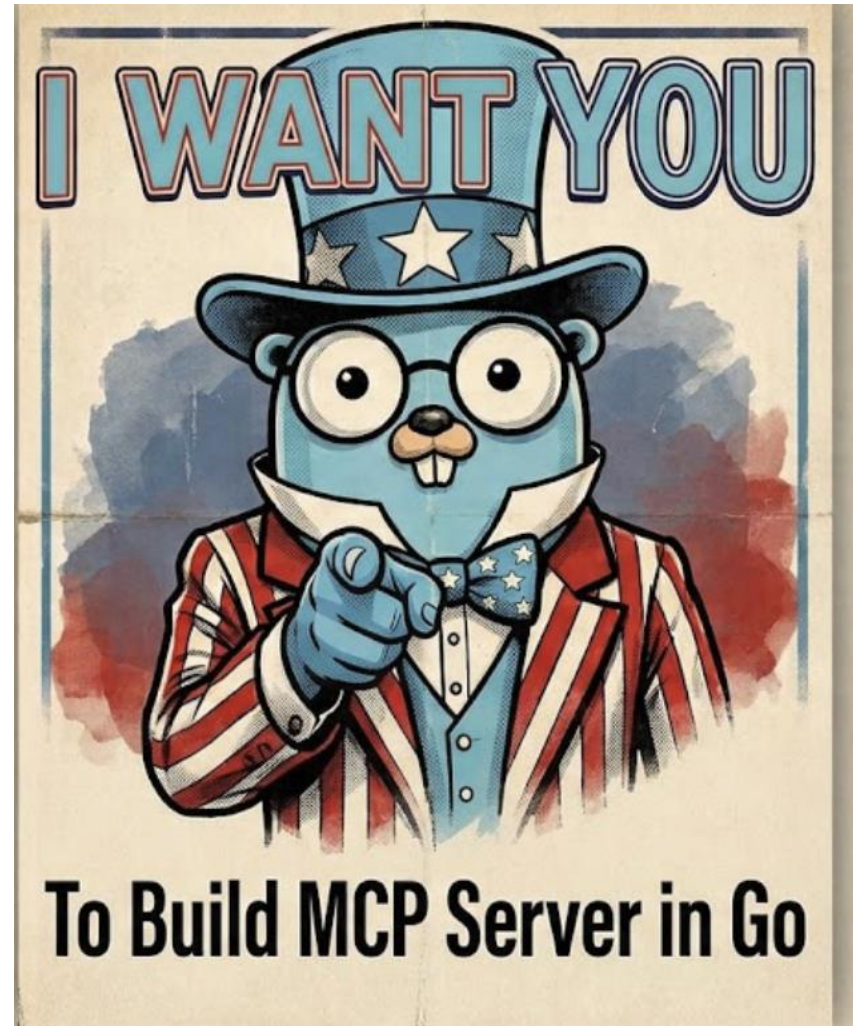
Decreased supply-chain risks

Faster startup time and tool registration

Cross-platform executables – choose features you bundle and how many tools

Faster iteration (and compile time) than Rust—especially for Python developers.

Mature, cross-platform libraries with lots of goodies!



A Minimal Go template

Library Selection

- Official Go SDK (<https://github.com/modelcontextprotocol/go-sdk>)
- Use SPF13 (Cobra/Viper/pflag) for command-line handling



Code Structure

```
internal/<app>/
config.go    # configuration and validation
client.go    # API, SDK, gRPC, or subprocess access
schemas.go   # typed inputs and domain validation
results.go   # limits, projection, and output formatting
```

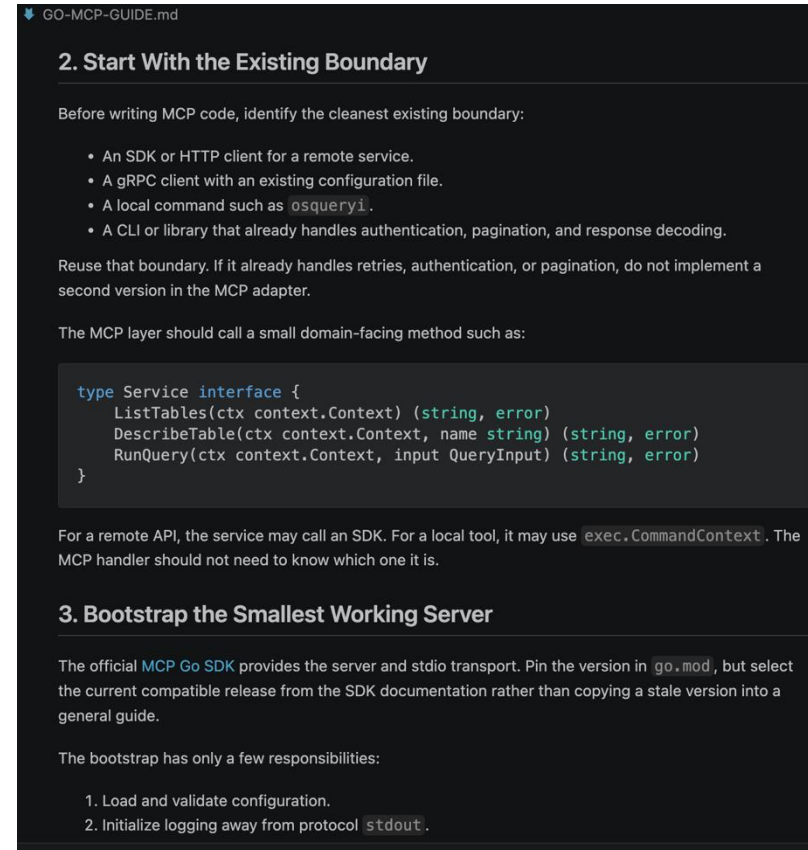
```
cmd/<app>-mcp/
main.go      # MCP bootstrap and stdio transport
tools.go     # tool schemas and handlers
```

```
cmd/<app>-cli/ # optional: deterministic human/automation interface
tools/        # optional: smoke and agent-driven tests
```



Create a PLAN.md based on GO-MCP-GUIDE.md

1. Decouple your MCP from internal libraries
2. Create a simple STDIO server with config and logging/testing from the start
3. Use progressive disclosure for tools (the ladder) using JSON Schema with tool descriptions
4. Implement guardrails based on failures and iterate on tools
5. Add additional operations and capabilities as needed



<https://github.com/mdfranz/dc-state-of-stack-2026-go-mcp/blob/main/refs/GO-MCP-GUIDE.md>

Lessons: instrument and iterate

Instrument from day one —
logging, observability, test
coverage

Add an LLM harness to
exercise common queries

Explore, then specialize &
optimize— let telemetry tell
you the next tool to build



```
Claude Code v2.1.267
Sonnet 5 with medium effort · Claude Pro
~/Documents/Documents - m4-mini-24gbi/github/osqueryi-mcp

▲ 2 MCP servers need authentication · run /mcp

> run the pydantic end to end tests

Searched for 1 pattern, read 1 file, ran 1 shell command

• Good, the binary and API keys are available. I'll run the full pydantic-ai suite against the default model (claude-haiku-4-5).

Ran 1 shell command

• The pydantic-ai end-to-end suite ran successfully against claude-haiku-4-5 — all tasks completed with no failures reported (overall totals: 18 requests, 126,656 tokens). Full log written to pydantic_ai_test.log, and traces/metrics were sent to Logfire.

* Baked for 1m 37s · done 9:55 PM

> can you summarize them briefly?

Ran 1 shell command

• That log file has accumulated history from many prior runs (going back to May/Aug), but here's today's run (2026-09-15 21:53-21:54), which is what I just executed:
```

Task	Requests	Tokens (in/out/total)	Elapsed
Structured discovery	3	7,251 / 1,235 / 8,486	11.8s
Single-table query	2	6,246 / 854 / 7,100	8.8s
Join-heavy workload	5	15,094 / 1,416 / 16,510	14.0s
Ambiguous classification query	8	91,052 / 3,508 / 94,560	34.2s
Overall	18	119,643 / 7,013 / 126,656	—

```
All 4 tasks completed with no failures, run against claude-haiku-4-5.
```

Lessons: Test Everything!

Unit tests — Both for CLI and MCP server

Direct JSON-RPC smoke test — no LLM required

Pydantic AI harness — real tools with a range of models.

Exploratory with Claude, Codex, OpenCode, etc.



Create virtuous, compounding feedback loops!

Climbing the tool ladder

Intent	Tool	Why
See the surface	list_tables	Cheapest inventory
Find relevant data	search_tables	Names + columns
Inspect one schema	describe_table	No row payloads
Learn shape + values	preview_table	Schema + sample
Query one table	query_table	Validated, bounded
Join or advanced SQL	run_query	Flexible, riskiest

Prefer the narrowest tool that answers the question.

Strongly typed MCP Servers in Go

Create dedicated
structs for each
tool and argument

Define and enforce
parameters

```
// Tool Arguments
type ListIndicesArgs struct {
    Pattern string `json:"pattern,omitempty" jsonschema:"Optional index pattern to filter by (e.g. logs-*)"`
}

type SearchArgs struct {
    Index string `json:"index" jsonschema:"The index pattern to search (e.g. logs-* or .alerts-security.alerts-default)"`
    Query any    `json:"query" jsonschema:"Elasticsearch JSON query DSL encoded as a string. Prefer exact term filters; av"`
}

type ClusterHealthArgs struct {
    Level string `json:"level,omitempty" jsonschema:"Optional detail level: cluster (default), indices, or shards"`
}

type LookupDomainArgs struct {
    Domain string `json:"domain" jsonschema:"The exact domain name to look up (e.g. connectivity-check.ubuntu.com)"`
}

type LookupIPArgs struct {
    IP string `json:"ip" jsonschema:"The IP address to look up (e.g. 8.8.8.8)"`
}
```

See <https://github.com/mdfranz/elastic-security-mcp/blob/main/internal/elasticsearch/tools.go>

Guard Out — bound the query

Restrict index and scope to prevent expensive, unbounded scans

Use **aggregation queries** with sane defaults to **get the shape of the data**

Require filters — and always bound the time range (and mind storage tiers)

Per-tool timeouts — surface partial results, relative to data you need

Validate data objects (indices, tables)



Guard In — bound the response

Cap rows and payloads returned to the LLM. Annotate objects when truncated.

Select only specific fields or JSON slices for only what you need

Provide helpful **error message** to inform agent retry

Export separately — paginate to files, never bulk JSON into context, MCP supports async and progress



E2E testing with Pydantic AI

Test across multiple
Providers and Models

Normalize model queries
and catch hallucinations

Test tool descriptions—
use evals.

Easy integration with
Logfire for token, turn,
and exception monitoring

```
SYSTEM_PROMPT = """
You are an Elastic Security expert.
Investigate Elasticsearch
```

```
Prefer the narrow, fast-path over the
cached history lookups, search for
search_security_alerts for a specific
broader network/endpoint if the narrow
else fits. Tool results prefixed with a cache indicator mean the answer was
served from (or written to) cache rather than freshly queried.
```

CRITICAL: After using tools, always provide a detailed final answer that includes:

1. Summary of what you discovered or found
2. Which specific tools you used and why each one
3. Key findings from the data (include sample rows if relevant)
4. Your analysis and conclusions

```
transport = StdioTransport(command=server_path, args=[], env=dict(os.environ))
async with MCPToolset(transport, max_retries=3) as server:
    agent = Agent(
        model_name,
        toolsets=[server],
        capabilities=[hooks],
        system_prompt=SYSTEM_PROMPT,
    )

    tasks = [*TASKS, build_export_task()]

    try:
        for title, prompt in tasks:
            stats.reset_task()
            start = time.perf_counter()
            logger.info("\n" + "=" * 20)
            logger.info(f" TASK: {title}")
            logger.info("=" * 20)
```

Agentsview for local analysis of Coding Agents

Local agentic
analytics for Claude,
Codex, and
OpenCode sessions

Tool sequence,
arguments, and
outcomes per task

Export sessions
along your .git history
to track improvement!

The screenshot displays the AgentsView application interface. At the top, there's a navigation bar with 'AgentsView', 'All Projects', and 'Sessions' tabs. A search bar is present. Below this, a session titled 'osquery_talk' is selected, showing a Claude agent's activity from August 13, 2025. The session details include a SQL query, its output, and tool calls like 'mcp__osqueryi-mcp__run_query'. Below the session details, an 'Assistant' section provides a summary of the analysis, noting that no malware or unauthorized persistence was found, but three items are worth attention. One item is 'Orphaned Wireshark ChmodBPF daemon', which is described as root persistence for software you uninstalled. A table shows the details of this daemon, including its path, mode, owner, group, and members.

sql: SELECT (SELECT global_state FROM alf) AS alf_firewall, (SELECT COUNT(*) FROM alf_exceptions) AS fw_exceptions, (SELECT COUNT(*) FROM homebrew_packages) AS brew_pkgs, (SELECT COUNT(*) FROM vscode_extensions) AS vscode_ext, (SELECT COUNT(*) FROM npm_packages) AS npm_pkgs, (SELECT COUNT(*) FROM python_packages) AS py_pkgs, (SELECT version FROM os_version) AS macos, (SELECT COUNT(*) FROM crashes) AS crashes

> output {"truncated":false,"results":[{"alf_firewall":"0","brew_pkgs":"129","crashes":"3","fw_exceptions":"7"}]}

> mcp__osqueryi-mcp__run_query 16.5s

> mcp__osqueryi-mcp__run_query 34.3s

Assistant Swept the persistence, inventory, and posture surfaces. **No malware, no unauthorized persistence.** But three things are worth your attention, and one is a genuine loose end.

Worth acting on

1. Orphaned Wireshark ChmodBPF daemon — root persistence for software you uninstalled

/Library/LaunchDaemons/org.wireshark.ChmodBPF.plist is active with RunAtLoad=1, executing a root-owned binary at every boot:

/Library/Application Support/Wireshark/ChmodBPF/ChmodBPF (uid 0, 0755, mtime 2025-01-08)

But /Applications/Wireshark.app **does not exist** — I checked it directly and got no row. The app is gone; its privileged boot item isn't.

It's still doing its job, which is the part that matters:

Check	Result
/dev/bpf* mode	0660, owner root, group 101
group 101	access_bpf
members	root, mdfranz

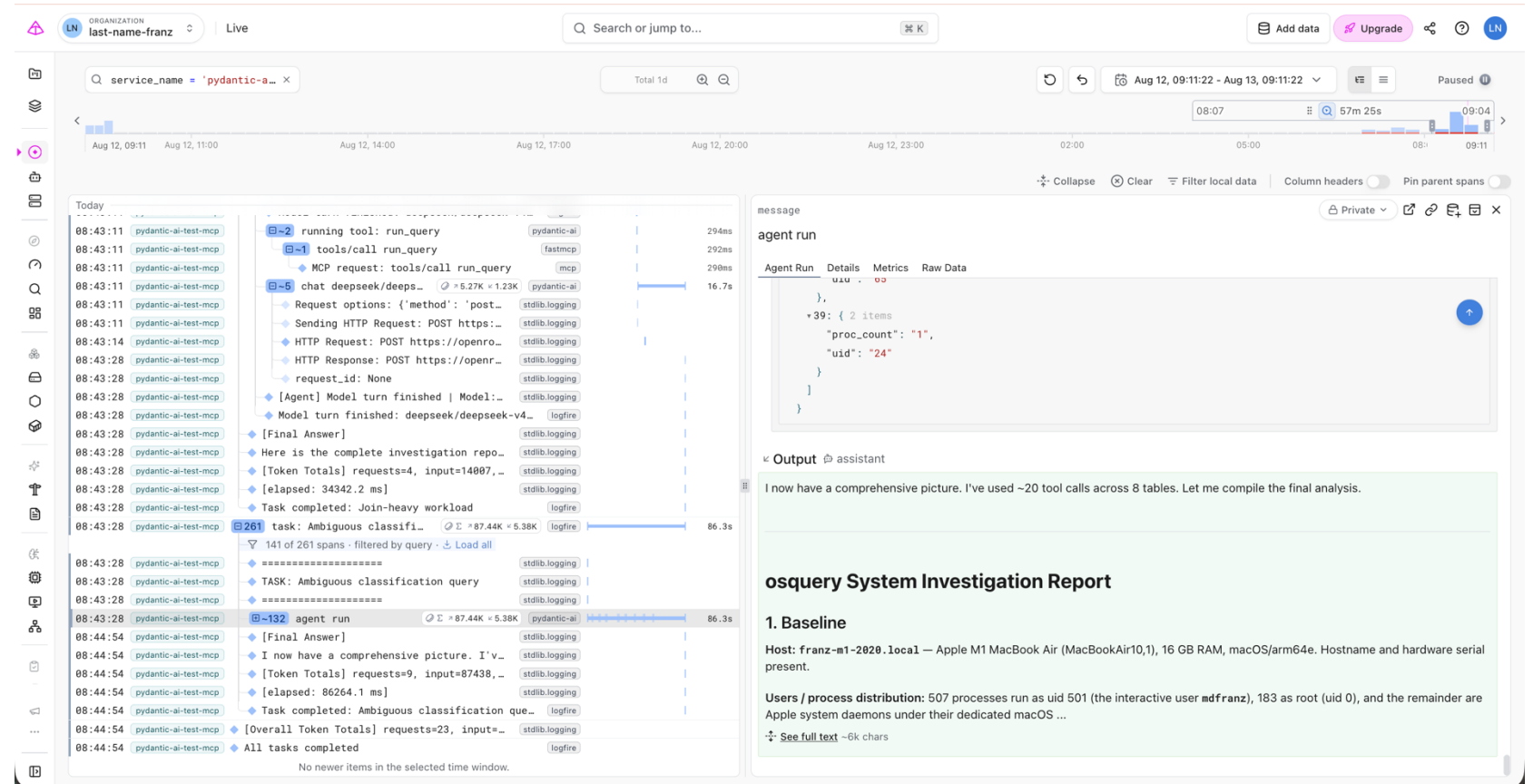
<https://github.com/kenn-io/agentsview> or <https://agenstview.io>

Logfire for Agent and MCP Observability!

OpenTelemetry traces
across model calls,
tool calls, and spans

Drop in libraries for
most frameworks—and
infra too!

Query able after the
fact, with a built-in MCP
server



<https://pydantic.dev/logfire>

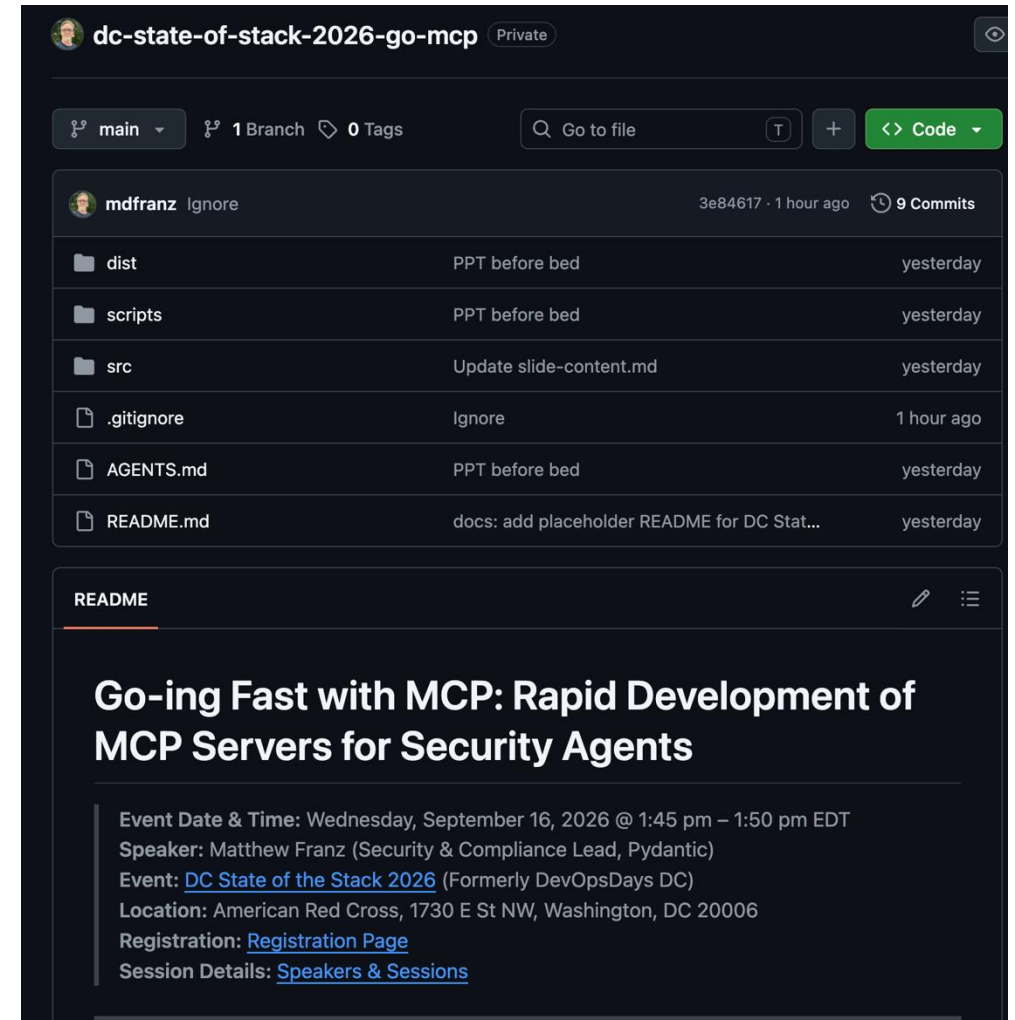
For more info

osqueryi-mcp — starter server, smoke harnesses, framework examples

elastic-security-mcp — structured tools, raw DSL fallback, caching, export

mcp-threat-hunt-aug-2026 — the long-form companion talk

<https://github.com/mdfranz/dc-state-of-stack-2026-go-mcp>



Takeaways and Thank You!

MCP (when implemented properly)
can make life easier for your agents

Be pragmatic, don't listen to the
haters!

Everything depends on your data and
operations

Have fun and keep learning.

